1. [20 pts] Consider a robot with position $\mathbf{p}(t) \in \mathbb{R}^3$ and orientation $R(t) \in SO(3)$ specified as follows:

$$\mathbf{p}(t) = \begin{bmatrix} t \\ 0 \\ 2t \end{bmatrix} \qquad R(t) = \begin{bmatrix} \cos(t\pi/3) & 0 & -\sin(t\pi/3) \\ 0 & 1 & 0 \\ \sin(t\pi/3) & 0 & \cos(t\pi/3) \end{bmatrix} \tag{1}$$

Determine the linear and angular velocity of the robot at time $t = 1$ with coordinates expressed in the robot frame.

$$\dot{T}(t) = T(t)\hat{\xi}(t)$$

$$T^{-1}(t)\dot{T}(t) = \hat{\xi}(t)$$

$$\begin{bmatrix} R^T & -R^T p \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} \dot{R} & \dot{p} \\ 0^T & 0 \end{bmatrix} = \hat{\xi}(t)$$

$$\dot{p}(t) = [1, 0, 2]^T \qquad p(t) = [t, 0, 2t]^T$$

$$\dot{R}(t) = \begin{bmatrix} -\frac{\pi}{3}\sin(\frac{t\pi}{3}) & 0 & -\frac{\pi}{3}\cos(\frac{t\pi}{3}) \\ 0 & 0 & 0 \\ \frac{\pi}{3}\cos(\frac{t\pi}{3}) & 0 & -\frac{\pi}{3}\sin(\frac{t\pi}{3}) \end{bmatrix} \qquad R(t) = \begin{bmatrix} \cos(t\pi/3) & 0 & -\sin(t\pi/3) \\ 0 & 1 & 0 \\ \sin(t\pi/3) & 0 & \cos(t\pi/3) \end{bmatrix}$$

$$\hat{\xi}(1) = T^{-1}(1)\dot{T}(1) = \begin{bmatrix} R^T(1) & -R^T(1)p(1) \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} \dot{R}(1) & \dot{p}(1) \\ 0^T & 0 \end{bmatrix}$$

$$R^T(1) = \begin{bmatrix} \cos(\frac{\pi}{3}) & 0 & -\sin(\frac{\pi}{3}) \\ 0 & 1 & 0 \\ \sin(\frac{\pi}{3}) & 0 & \cos(\frac{\pi}{3}) \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & 0 & -\frac{\sqrt{3}}{2} \\ 0 & 1 & 0 \\ \frac{\sqrt{3}}{2} & 0 & \frac{1}{2} \end{bmatrix} \qquad p(1) = [1, 0, 2]^T$$

$$\dot{R}(1) = \begin{bmatrix} -\frac{\pi}{3}\sin(\frac{\pi}{3}) & 0 & -\frac{\pi}{3}\cos(\frac{\pi}{3}) \\ 0 & 0 & 0 \\ \frac{\pi}{3}\cos(\frac{\pi}{3}) & 0 & -\frac{\pi}{3}\sin(\frac{\pi}{3}) \end{bmatrix} = \begin{bmatrix} -\frac{\pi\sqrt{3}}{6} & 0 & -\frac{\pi}{6} \\ 0 & 0 & 0 \\ \frac{\pi}{6} & 0 & -\frac{\pi\sqrt{3}}{6} \end{bmatrix} \qquad \dot{p}(1) = [1, 0, 2]^T$$

$$\hat{\xi} = \begin{bmatrix} \begin{bmatrix} \frac{1}{2} & 0 & -\frac{\sqrt{3}}{2} \\ 0 & 1 & 0 \\ \frac{\sqrt{3}}{2} & 0 & \frac{1}{2} \end{bmatrix}^T & -\begin{bmatrix} \frac{1}{2} & 0 & -\frac{\sqrt{3}}{2} \\ 0 & 1 & 0 \\ \frac{\sqrt{3}}{2} & 0 & \frac{1}{2} \end{bmatrix}^T \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} \\ 0 \quad 0 \quad 0 & 1 \end{bmatrix} \begin{bmatrix} \begin{bmatrix} \frac{\pi\sqrt{3}}{6} & 0 & -\frac{\pi}{6} \\ 0 & 0 & 0 \\ \frac{\pi}{6} & 0 & -\frac{\pi\sqrt{3}}{6} \end{bmatrix} & \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} \\ 0 \quad 0 \quad 0 & 0 \end{bmatrix}$$

$$\rightarrow \left[-\frac{1+2\sqrt{3}}{2}, 0, -\frac{-\sqrt{3}+2}{2}\right]$$

$$\Gamma = \begin{bmatrix} 0 & 0 & -\frac{\pi}{3} & \frac{1+2\sqrt{3}}{2} \\ 0 & 0 & 0 & 0 \\ -\frac{\pi}{6} & 0 & 0 & \frac{-\sqrt{3}+2}{2} \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\Rightarrow \hat{\omega} = \begin{bmatrix} 0 & 0 & -\frac{\pi}{3} \\ 0 & 0 & 0 \\ \frac{\pi}{3} & 0 & 0 \end{bmatrix}$$

$$\Rightarrow \omega = \begin{bmatrix} 0 \\ -\frac{\pi}{3} \\ 0 \end{bmatrix}$$

and $$V = \begin{bmatrix} \frac{1+2\sqrt{3}}{2} \\ 0 \\ \frac{-\sqrt{3}+2}{2} \end{bmatrix}$$

2. [20 pts] Consider a stereo camera with the following position, orientation, baseline, and intrinsic parameter matrix:

$$ {}^{w}\mathbf{p}_{c} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}, \quad {}^{w}_{c}R = \begin{bmatrix} \sqrt{3}/2 & 1/2 & 0 \\ -1/2 & \sqrt{3}/2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad b = 0.5, \quad K = \begin{bmatrix} 10 & 0 & 100 \\ 0 & 10 & 100 \\ 0 & 0 & 1 \end{bmatrix}. \tag{2} $$

The camera observes a point $\mathbf{s}$ with optical-frame pixel coordinates $\mathbf{z}_L = [102,\ 85]^\top$ in the left image and $\mathbf{z}_R = [99,\ 85]^\top$ in the right image. Determine the world-frame coordinates of the point $\mathbf{s}$.

$z_L = \begin{bmatrix} 102 \\ 85 \end{bmatrix} = \begin{bmatrix} u_L \\ v_L \end{bmatrix}$, $z_R = \begin{bmatrix} 99 \\ 85 \end{bmatrix} = \begin{bmatrix} u_R \\ v_R \end{bmatrix}$ $K = \begin{bmatrix} 10 & 0 & 100 \\ 0 & 10 & 100 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} f_{su} & f_{s\theta} & c_u \\ 0 & f_{sv} & c_v \\ 0 & 0 & 1 \end{bmatrix}$

$d = u_L - u_R = 102 - 99 = 3$

$$ \begin{bmatrix} u_L \\ v_L \\ d \end{bmatrix} = \begin{bmatrix} f_{su} & 0 & c_u & 0 \\ 0 & f_{sv} & c_v & 0 \\ 0 & 0 & 0 & f_{su} b \end{bmatrix} \frac{1}{z} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} $$

$$ \begin{bmatrix} 102 \\ 85 \\ 3 \end{bmatrix} = \begin{bmatrix} 10 & 0 & 100 & 0 \\ 0 & 10 & 100 & 0 \\ 0 & 0 & 0 & 5 \end{bmatrix} \frac{1}{z} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} $$

$102 = 10\frac{x}{z} + 100\cdot\frac{z}{z}$

$85 = 10\frac{y}{z} + 100\frac{z}{z}$

$3 = \frac{5}{z} \Rightarrow z = \frac{5}{3}$

$102 = 10\frac{x}{5/3} + 100 \Rightarrow x = \frac{2}{6} = \frac{1}{3}$

$85 = 10\frac{y}{5/3} + 100 \Rightarrow y = \frac{-15}{6} = -\frac{5}{2}$

$z = \frac{5}{3}$

$s_c = \left[\frac{1}{3}, -\frac{5}{2}, \frac{5}{3}\right]$

$s = Rs + p$

$$ = \begin{bmatrix} \sqrt{3}/2 & 1/2 & 0 \\ -1/2 & \sqrt{3}/2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1/3 \\ -5/2 \\ 5/3 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} $$

$$ = \begin{bmatrix} \frac{2-\sqrt{3}}{4\sqrt{3}} \\ \frac{-14-15\sqrt{3}}{12} \\ \frac{5}{3} \end{bmatrix} $$

3. [30 pts] Consider two sets of points $\{\mathbf{z}_i\}$ and $\{\mathbf{m}_i\}$ with coordinates provided in Table 1 and Table 2, respectively. Determine the translation $\mathbf{p} \in \mathbb{R}^3$ and rotation $\mathbf{R} \in SO(3)$ that aligns the points such that $\mathbf{R}\mathbf{z}_i + \mathbf{p} = \mathbf{m}_i$ for all $i$.

Table 1: Coordinates $\mathbf{z}_i$ of eight points $i = 1, \ldots, 8$.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0.2785 | 0.1576 | 0.8003 | 0.7922 | 0.8491 | 0.7431 | 0.7060 | 0.0971 |
| 0.5469 | 0.9706 | 0.1419 | 0.9595 | 0.9340 | 0.3922 | 0.0318 | 0.8235 |
| 0.9575 | 0.9572 | 0.4218 | 0.6557 | 0.6787 | 0.6555 | 0.2769 | 0.6948 |

Table 2: Coordinates $\mathbf{m}_i$ of eight points $i = 1, \ldots, 8$.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0.5391 | 0.2632 | 0.4950 | 0.2118 | 0.2699 | 0.5128 | 0.4039 | 0.1199 |
| -0.5638 | -0.7963 | 0.2746 | -0.1363 | -0.1258 | 0.0386 | 0.3606 | -0.6349 |
| 1.8256 | 1.5871 | 1.6287 | 1.4197 | 2.1640 | 1.1055 | 0.5774 | 1.1945 |

$$\bar{m} = \frac{\sum_i w_i m_i}{\sum_i w_i} \qquad \bar{z} = \frac{\sum_i w_i z_i}{\sum w_i}$$

$$\delta m_i = m_i - \bar{m} \qquad \delta z_i = z_i - \bar{z}$$

$$Q = \sum_i w_i \, \delta m_i \, \delta z_i^T = U \Sigma V^T$$

$$\Rightarrow R = U \begin{bmatrix} 1 & & \\ & \ddots & \\ & & \det(UV^T) \end{bmatrix} V^T$$

$$\Rightarrow p = \bar{m} - R\bar{z}$$

```python
wi = np.array([1, 1, 1, 1, 1, 1, 1, 1])

# Compute weighted centroids
w_sum = np.sum(wi)

m_bar = np.sum(wi * mi, axis=1) / w_sum  # Weighted mean of mi
z_bar = np.sum(wi * zi, axis=1) / w_sum  # Weighted mean of zi

# Compute deviations from centroids
delta_mi = mi - m_bar[:, np.newaxis]  # Each column is mi - m_bar
delta_zi = zi - z_bar[:, np.newaxis]  # Each column is zi - z_bar

# Compute Q matrix
Q = np.dot(delta_mi * wi[np.newaxis, :], delta_zi.T)

# Perform SVD
U, Sigma, Vt = np.linalg.svd(Q)

det_term = np.linalg.det(np.dot(U, Vt))  # Compute det(UV^T)
D = np.eye(3)  # Identity matrix (3x3)
D[2, 2] = det_term  # Modify the last diagonal entry

# Compute R
R = np.dot(U, np.dot(D, Vt))

# Compute translation vector p
p = m_bar - np.dot(R, z_bar)
```

```
Rotation Matrix:
[[ 0.09291591 -0.69223203  0.71566853]
 [ 0.83151082 -0.3414247  -0.43819965]
 [ 0.54768275  0.63580185  0.54387463]]
Translation:
[ 0.24198206 -0.1626525   0.39325011]
```

4. [30 pts] Consider two cameras observing the same scene from different view points. Each camera observes the same set of points and returns their normalized pixel coordinates, provided in Table 3 and Table 4. The intrinsic parameter matrices of both cameras are assumed to be identity. Determine the rotation and translation of the second camera reference frame with respect to the first camera reference frame.

Table 3: Normalized pixel coordinates returned by camera 1.

$\underline{z}_{t,i} :=$

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0.2953 | 0.1658 | 0.3040 | 0.1492 | 0.1247 | 0.4638 | 0.6996 | 0.1004 |
| -0.3088 | -0.5017 | 0.1686 | -0.0960 | -0.0581 | 0.0349 | 0.6246 | -0.5315 |

Table 4: Normalized pixel coordinates returned by camera 2.

$\underline{z}_{t+1,i} :=$

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0.2909 | 0.1647 | 1.8975 | 1.2081 | 1.2510 | 1.1337 | 2.5496 | 0.1398 |
| 0.5712 | 1.0140 | 0.3364 | 1.4632 | 1.3761 | 0.5984 | 0.1150 | 1.1851 |

$K_t = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, $K_{t+1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

$\underline{y}_{t,i} = K_t^{-1} \underline{z}_{t,i}$ and $\underline{y}_{t+1,i} = K_{t+1}^{-1} \underline{z}_{t+1,i}$

$\bar{y}_i = \text{vec}\left(\underline{y}_{t,i} \cdot \underline{y}_{t+1,i}^T\right)$

$\bar{y} = [\bar{y}_1 \cdots \bar{y}_8]^T$

$\bar{y} e = 0$

$\Rightarrow$ find eigenvector $e$ associated with $0$ eigenvalue

$E = \text{vec}(e)$

Decompose $E = U \,\text{diag}(\sigma, \sigma, 0)\, V^T$

We guess that

$(\hat{p}, R) = \left(U R_z\left(\frac{\pi}{2}\right) \text{diag}(\sigma, \sigma, 0)\, U^T,\ U R_z^T\left(\frac{\pi}{2}\right) V^T\right)$ or

$(\hat{p}, R) = \left(U R_z\left(-\frac{\pi}{2}\right) \text{diag}(\sigma, \sigma, 0)\, U^T,\ U R_z^T\left(-\frac{\pi}{2}\right) V^T\right)$

```
k1 = np.eye(3)
k2 = np.eye(3)

y1i = np.linalg.inv(k1) @ z1i  # y1i = K1^-1 * z1i
y2i = np.linalg.inv(k2) @ z2i  # y2i = K2^-1 * z2i

yi_vecs = []

# Loop over each column of y2i
for i in range(y2i.shape[1]):

    outer_product = np.multiply(y1i[:, i].reshape((3,1)), y2i[:, i].reshape((3,1)).T)  # Compute product

    vectorized = np.ravel(outer_product, order='F')  # Flatten to a vector

    yi_vecs.append(vectorized)  # Append to the list

# Stack the column vectors into a matrix
y_bar = np.array(yi_vecs)

U, S, Vt = np.linalg.svd(y_bar)

e = Vt[-1, :]  # Last row of V^T (i.e., last column of V)

E = e.reshape(3, 3).T

# Compute the SVD decomposition of E
U, Sigma, Vt = np.linalg.svd(E)

Sigma = np.diag(Sigma)

# To ensure the determinant is +1
Vt[-1, :] *= -1
Sigma[-1, -1] *= -1
```

```
# Define the 90-degree rotation matrix Rz(pi/2) and Rz(-pi/2)
Rz_pos = np.array([
    [0, -1, 0],
    [1,  0, 0],
    [0,  0, 1]
])

Rz_neg = np.array([
    [0,  1, 0],
    [-1, 0, 0],
    [0,  0, 1]
])

# Compute the two possible solutions for (p_hat, R)
P_hat_1 = U @ Rz_pos @ Sigma @ U.T
R1 = U @ Rz_pos.T @ Vt

P_hat_2 = U @ Rz_neg @ Sigma @ U.T
R2 = U @ Rz_neg.T @ Vt

def vee_map(p_hat):
    """
    Recovers the vector p from its skew-symmetric matrix representation p_hat.
    """
    p1 = p_hat[2, 1]  # Extract p1 from (3,2) position
    p2 = p_hat[0, 2]  # Extract p2 from (1,3) position
    p3 = p_hat[1, 0]  # Extract p3 from (2,1) position

    return np.array([p1, p2, p3])


p1 = vee_map(P_hat_1)
p2 = vee_map(P_hat_2)
```

```
Guess 1:
Translation:
[ 0.00915337 -0.02884154  0.7064024 ]
Rotation Matrix:
[[ 0.2888588  -0.55214875  0.78210763]
 [ 0.74425842 -0.38434779 -0.54621989]
 [ 0.60219598  0.73987062  0.29991912]]

Guess 2:
Translation:
[-0.00914229  0.02883801 -0.70640226]
Rotation Matrix:
[[-0.27389008  0.57161019 -0.77346365]
 [-0.79113502  0.32340161  0.51915005]
 [ 0.54689085  0.75410423  0.36364435]]
```